

Recovering drift fields from random walks

Six estimators, one set of walkers, and where each one wins

The dfi package • 46 tests (45 pass, 1 skips) • every number reproduced by a numbered script

The question. Particles move by $dx = b(x) dt + \sqrt{2D} dW$. From their observed motion, recover the vector field b . Over one recorded step the random displacement is about 30 times the drift displacement (about 1000 times in power), so every method is an exercise in averaging away a much larger random term without averaging away the field.

What was built. A simulator, six estimators of increasing structure, and one measurement protocol that runs all of them on the *same* walkers, so that differences are differences between methods and not between experiments.

Three results. (i) A network trained across many simulated fields beats every classical estimator by roughly 25–40% — and loses to the simplest kernel on fields from a generator it never saw. (ii) At equilibrium, *static positions* carry more about the drift than the increments do, decisively so in 5D. (iii) Static positions are *exactly* blind to circulation: adding a rotating component to the drift leaves every snapshot-based estimate unchanged to the last bit.

1 The problem

1.1 The dynamics

The overdamped Langevin equation

$$dx = b(x) dt + \sqrt{2D} dW \quad (1)$$

models a particle in a force field b with thermal agitation of strength D (the diffusion coefficient). W is a d -dimensional Wiener process (Brownian motion): its increments over disjoint time intervals are independent and Gaussian, with

$$\mathbb{E}[dW_i] = 0, \quad \mathbb{E}[dW_i dW_j] = \delta_{ij} dt, \quad \text{i.e.} \quad W(t + \Delta t) - W(t) \sim \mathcal{N}(0, \Delta t I_d). \quad (2)$$

The noise term $\sqrt{2D} dW$ therefore has zero mean and variance $2D dt$ per component; for $b = 0$ this is free diffusion with mean squared displacement $2dDt$. Its standard deviation grows like $\sqrt{dt}$, much faster than the drift's dt for small dt — which is the whole difficulty below.

The inverse problem, recovering b from observed motion, appears in single-particle tracking, active matter, ecology, and anywhere a stochastic differential equation is fitted to a trajectory. Two kinds of data support two fundamentally different answers.

1.2 Trajectories: the drift is a conditional mean

Positions recorded in time order at spacing Δt . Integrating (1) over one step, $\Delta x = b(x)\Delta t + \sqrt{2D} \Delta W + O(\Delta t^{3/2})$, so

$$\mathbb{E}[\Delta x | x] = b(x) \Delta t + O(\Delta t^2), \quad \text{Cov}[\Delta x | x] = 2D\Delta t I + O(\Delta t^2). \quad (3)$$

This is the Kramers–Moyal relation. It turns each recorded step into a noisy sample of the field at the point the step started from:

$$y_i \equiv \frac{\Delta x_i}{\Delta t} = b(x_i) + \eta_i, \quad \eta_i \sim \mathcal{N}\left(0, \frac{2D}{\Delta t} I\right). \quad (4)$$

Estimating b is therefore a regression of y on x . The noise η grows as Δt shrinks: in the 2D settings used here ($D = 0.3$, $\Delta t \approx 10^{-3}$, $\text{RMS}|b| \approx 1$) its magnitude is 32 times the drift's. The six estimators differ only in how they pool neighbouring samples to beat it down.

1.3 Snapshots: what the stationary density can and cannot tell

Positions *without* time order, drawn from the stationary density ρ_{ss} . The density obeys the Fokker–Planck equation

$$\partial_t \rho = -\nabla \cdot J, \quad J = b \rho - D \nabla \rho, \quad (5)$$

where J is the probability current. In the stationary state $\nabla \cdot J_{ss} = 0$. Solving the definition of J for b gives an identity valid for *any* drift:

$$b = \underbrace{D \nabla \log \rho_{ss}}_{\text{visible in snapshots}} + \underbrace{J_{ss}/\rho_{ss}}_{\text{invisible in snapshots}}. \quad (6)$$

Snapshots determine ρ_{ss} , hence the first term (the *score* $\nabla \log \rho_{ss}$, times D). They say nothing about the second: any field c with $\nabla \cdot (c \rho_{ss}) = 0$ can be added to b without changing ρ_{ss} , so infinitely many drifts produce the same snapshots. Only when the current vanishes — *detailed balance*, i.e. thermodynamic equilibrium — is $b = D \nabla \log \rho_{ss}$ a pure gradient, fully determined by positions alone.

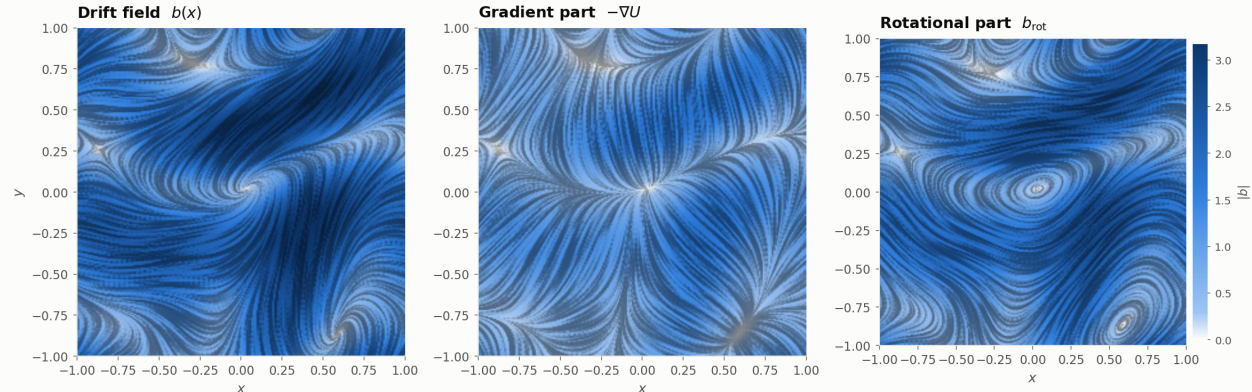
The fields used here make this exact. Every drift is built as

$$b = -\nabla U + \omega A \nabla U, \quad A^\top = -A, \quad (7)$$

from one scalar potential U . Substituting $\rho_{ss} = e^{-U/D}/Z$ into the current gives $J_{ss} = \omega \rho_{ss} A \nabla U$, and its divergence vanishes term by term: $\nabla \cdot (A \nabla U) = A_{ij} \partial_i \partial_j U = 0$ (an antisymmetric matrix contracted with a symmetric Hessian), and $\nabla U \cdot A \nabla U = 0$ (an antisymmetric quadratic form). So $\rho_{ss} \propto e^{-U/D}$ for *every* ω : the rotational part $\omega A \nabla U$ drives a steady circulating current $J_{ss} \neq 0$ yet leaves the density untouched, and the score recovers only the gradient part,

$$D \nabla \log \rho_{ss} = -\nabla U = b_{\text{grad}}. \quad (8)$$

Helmholtz decomposition -- random-grf-2D



$b = -\text{grad } U + \omega A \text{ grad } U$ with $\omega = 1.5$. The two components are orthogonal pointwise and the rotational one is divergence free, so ρ_{ss} is unchanged by ω .

Figure 1: A drift field (7) and its two parts. The rotational part (right) is divergence-free and everywhere orthogonal to $\nabla \rho_{ss}$, so ρ_{ss} is identical for every ω : a snapshot dataset cannot distinguish a system with no circulation from one with a great deal of it.

The identifiability gap is a property of the data, not of the estimators. From snapshots, only the gradient part of b is recoverable, whatever the method; from trajectories, all of b is. The comparisons below are read with this limitation in mind.

2 Setup and measurement protocol

Fields. U is a Gaussian random field on the torus $[-1, 1]^d$ with a tunable correlation length, and b is built from it by (7), so ω dials circulation without touching ρ_{ss} . Six other generators (sparse wells, cellular band-pass patterns, Perlin noise, power-law spectra, terraces, an egg-carton lattice) are used only to test generalisation.

Simulation. Euler–Maruyama with sub-stepping: the integrator runs at Δt /substeps while positions are recorded at Δt , so that an $O(\Delta t)$ bias in an estimator is not confused with an $O(\Delta t)$ error of the integrator. Trajectories are cached by configuration, so every estimator sees bit-identical walkers.

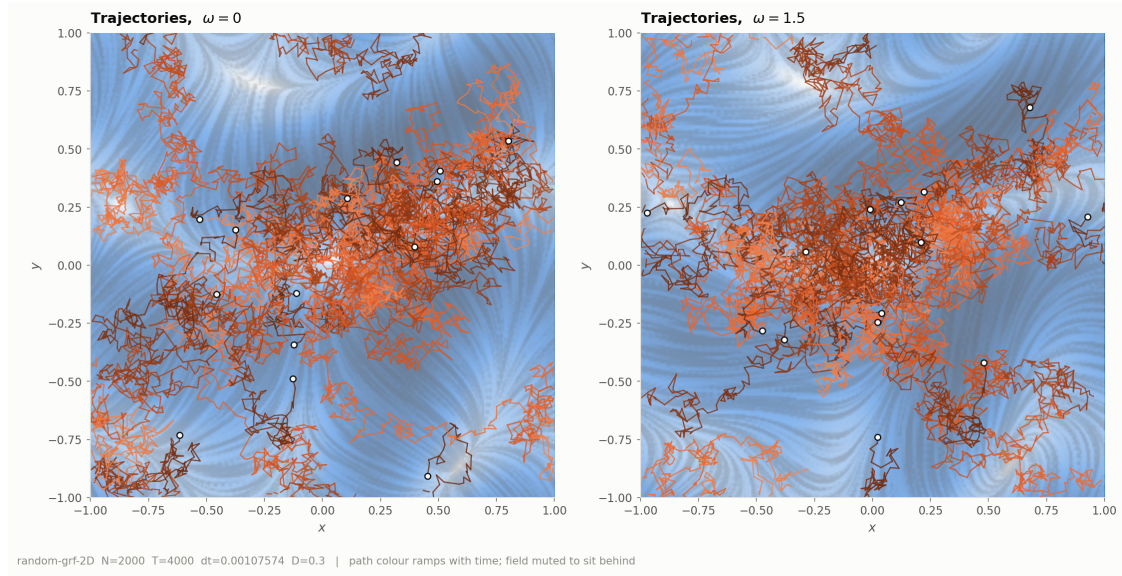


Figure 2: What the estimators are given: 2000 walkers $\times$ 4000 steps in the same field, without and with circulation ($\omega = 0$ and 1.5), paths coloured by time. The drift is invisible in any single step and appears only after averaging thousands of them. The circulation on the right is recoverable from the increments and, by (6), absent from the positions.

Error metric. Every estimate is scored against the true field at $n = 20\,000$ points x_j drawn from ρ_{ss} — where the walkers actually go, and so where the estimate will be used:

$$\text{nmse} = \left(\frac{\sum_j |\hat{b}(x_j) - b(x_j)|^2}{\sum_j |b(x_j)|^2} \right)^{1/2}. \quad (9)$$

It is 0 for a perfect estimate and exactly 1 for the trivial estimate $\hat{b} \equiv 0$, so 0.10 means “wrong by 10% of the field’s typical magnitude”, and anything near 1 means the method has learned essentially nothing. Lower is better.

Tuning. Every method chooses its own smoothing (bin size, bandwidth, basis size, training length) by cross-validation grouped *by walker*, never by sample. Splitting increments at random lets a held-out point sit one time step away from its training neighbours, and the criterion then rewards a smoothing scale far too small (§4.6).

3 Six estimators, from simplest to most structured

	method	data	what it assumes
1	binned Kramers–Moyal	increments	b is constant inside a cell
2	kernel regression (Nadaraya–Watson, local-linear)	increments	b is smooth on one length scale
3	basis projection (SFI)	increments	b lies in a Fourier basis
4	neural network (MLP, CNN) fitted to one dataset	increments	smoothness from architecture + early stopping
4b	amortised CNN, trained across 3000 fields	increments, binned	what fields from this generator look like
5	KDE score, denoising score matching	positions only, plus D	equilibrium (gradient drift)

3.1 Methods 1–4: least squares on the increments

All four trajectory estimators minimise the same squared loss over different classes of functions. That loss is not an arbitrary choice; three properties make it the right one.

1. **It targets b without seeing b .** Since $\mathbb{E}[\eta | x] = 0$, for any candidate f

$$\mathbb{E} |y - f(x)|^2 = \mathbb{E} |b(x) - f(x)|^2 + \mathbb{E} |\eta|^2, \quad (10)$$

the error against the unknown field plus a constant. Minimising the loss on the observed y minimises the error against b ; its minimiser over all functions is the conditional mean $\mathbb{E}[y | x] = b(x)$.

2. **It is maximum likelihood.** The Euler transition density is Gaussian, $p(x_{k+1}|x_k) \propto \exp(-|\Delta x_k - b(x_k)\Delta t|^2/4D\Delta t)$, so the log-likelihood of a trajectory is $-\frac{\Delta t}{4D} \sum_k |y_k - b(x_k)|^2 + \text{const.}$
3. **It has a readable training curve.** Because the noise term in (10) is about 1000 times the signal, the raw loss barely moves. The *gain over predicting zero*, $L(0) - L(f) = \mathbb{E}|b|^2 - \mathbb{E}|b - f|^2 = (1 - \text{nmse}^2) \mathbb{E}|b|^2$, isolates the part that does.

Losses of methods 1–4 (data $\{(x_i, y_i)\}_{i=1}^n, y_i = \Delta x_i/\Delta t$)

$$1. \text{ binning: } \hat{b}_c = \arg \min_{\beta} \sum_{i: x_i \in c} |y_i - \beta|^2 = \text{mean of the } y_i \text{ in cell } c$$

$$2. \text{ kernel (NW): } \hat{b}(x) = \arg \min_{\beta} \sum_i K_h(x_i - x) |y_i - \beta|^2$$

$$2. \text{ kernel (local-linear): } (\hat{b}(x), \hat{B}) = \arg \min_{\beta, B} \sum_i K_h(x_i - x) |y_i - \beta - B(x_i - x)|^2$$

$$3. \text{ SFI: } \hat{c} = \arg \min_c \sum_i \left| y_i - \sum_{|k| \leq K} c_k \phi_k(x_i) \right|^2, \quad \hat{b} = \sum_k \hat{c}_k \phi_k$$

$$4. \text{ neural network: } \hat{\theta} = \arg \min_{\theta} \frac{1}{n} \sum_i |y_i - f_{\theta}(x_i)|^2$$

K_h is a Gaussian kernel of bandwidth h ; ϕ_k are Fourier modes on the torus. The smoothing parameter of each — cell width, h , the number of modes K , the number of training steps — is chosen by the same squared loss on held-out walkers.

Binning returns one value for the whole cell; the local-linear kernel removes the first-order bias this carries; SFI (stochastic force inference) projects onto a global Fourier basis, which is exactly the right basis for these fields. The networks are an MLP with periodic input features, or a CNN that outputs the field on a grid; for the CNN the loss over millions of increments reduces exactly to a few lattice statistics, so it is trained full-batch with no sampling noise.

3.2 Method 4b: an amortised network, trained across fields

The per-dataset network must be refitted for every new dataset. The amortised one is trained *once* on simulated data and then only applied.

Training data. 3000 fields drawn from the generator with randomised parameters (correlation length 0.3–0.7; $\omega = 0$ for half, otherwise in $[-1.5, 1.5]$; D log-uniform in $[0.05, 1.5]$; 256 or 1024 walkers for 1000–4000 steps). Each dataset is summarised on a 64×64 lattice by the count N_c and the summed increments $Y_c = \sum_{i \in c} y_i$ per cell.

Inputs. Five channels built from N_c, Y_c and the noise level $\sigma^2 = 2D/\Delta t$, which is estimated from the data itself: a z-score $Y/(\sigma\sqrt{N})$, a shrunk mean $Y/(N + \lambda)$, and the data weight $N/(N + \lambda)$, with $\lambda = \sigma^2/\tau^2$. The shrunk mean is precisely the per-cell Bayesian estimate: if a cell's mean increment $\bar{Y} = Y/N \sim \mathcal{N}(b, \sigma^2/N)$

and the prior is $b \sim \mathcal{N}(0, \tau^2)$, the posterior mean is $Y/(N + \lambda)$. The network starts from that and learns the spatial coupling between cells.

Loss of method 4b (over training fields m , on the 64^2 lattice nodes c)

$$L(\theta) = \mathbb{E}_m \left[\frac{\sum_c w_c |f_\theta(X^{(m)})_c - b_c^{(m)}|^2}{\sum_c w_c |b_c^{(m)}|^2} \right], \quad w_c = 0.8 \rho_{ss}(c)/\bar{\rho} + 0.2.$$

$X^{(m)}$ are the input channels of training dataset m and $b^{(m)}$ its true field. Unlike every other method, the truth appears in this loss – but only for the simulated training fields. On a new dataset the network is applied without retraining and never sees b .

Why this loss. The minimiser of $\mathbb{E} \|f(X) - b\|^2$ over all functions is the posterior mean $\mathbb{E}[b | X]$ under the training distribution: the network is trained to be the *Bayes-optimal* estimator for fields drawn from its generator. That single fact explains both its advantage and its failure mode (§4.2). Dividing by $\sum_c w_c |b_c|^2$ makes every field count equally regardless of its amplitude; weighting by ρ_{ss} matches the evaluation metric (9), with a 20% floor so sparsely visited regions are not ignored. The network is a periodic U-Net with no normalisation layers (which would erase the physical amplitude), trained with the 8 symmetries of the square lattice applied to inputs and outputs, vector components included.

3.3 Method 5: the score, from static positions

By (8), at equilibrium $b = D s$ with $s = \nabla \log \rho_{ss}$. Estimating a score from samples is exactly what training a diffusion model does.

Losses of method 5 (data: positions $\{x_i\}$ drawn from ρ_{ss})

$$\text{denoising score matching: } L(\theta) = \mathbb{E}_{x, \sigma, \varepsilon} \left[\sigma s_\theta(x + \sigma \varepsilon, \sigma) + \varepsilon \right]^2$$

$$x \sim \text{data}, \quad \varepsilon \sim \mathcal{N}(0, I), \quad \log \sigma \sim \mathcal{U}[\log 0.01, \log 0.5]$$

$$\text{KDE score: } s_h(x) = \nabla \log \hat{\rho}_h(x), \quad \hat{\rho}_h(x) = \frac{1}{n} \sum_i \mathcal{N}(x; x_i, h^2 I)$$

$$\text{selection (both): } J(s) = \mathbb{E}_{x \sim \rho_{ss}} \left[\frac{1}{2} |s(x)|^2 + \nabla \cdot s(x) \right] \text{ on held-out positions}$$

Why these losses. Denoising score matching is again least squares onto a conditional mean. For a noised point $\tilde{x} = x + \sigma \varepsilon$, Tweedie’s formula gives $-\mathbb{E}[\varepsilon | \tilde{x}]/\sigma = \nabla \log \rho_\sigma(\tilde{x})$, where $\rho_\sigma = \rho_{ss} * \mathcal{N}(0, \sigma^2 I)$; so the minimiser of L is the score of a slightly blurred density, and σ plays the role of a bandwidth. The selection criterion J (Hyvärinen score matching) satisfies, after an integration by parts with no boundary terms on the torus, $J(s) = \frac{1}{2} \mathbb{E} |s - \nabla \log \rho_{ss}|^2 + \text{const}$: it ranks candidate scores by their true error using samples alone. It sets the KDE bandwidth h and the noise level σ at which the trained network is read out.

4 Results

How to read Table 1. Each entry is the nrmse (9): 0 is perfect, 1 is what predicting $\hat{b} \equiv 0$ scores. The first column is the reference configuration; each other column changes one thing, stated in the second header row, and the third row gives the data budget in recorded transitions. Trajectory and snapshot rows are different kinds of data on the same systems: the snapshot methods get independent positions matched in number to the transitions (capped at 500k), plus the true D . Bold marks the best method of each family in each column.

4.1 More structure beats more flexibility – until the structure is wrong

In the reference column the order is binning (0.293), kernel (0.126), network (0.100), Fourier basis (0.097). Each step buys accuracy by assuming more, and here the assumptions happen to be true: these fields *are* sums of

Table 1: Normalised error (9) of every method; lower is better.

	reference	few walkers	many walkers	strong noise	position error	3D	5D
changed	2D, $D = 0.3$	16 walkers	1024 walkers	$D = 1.0$	$\sigma_{\text{obs}} = 0.02$	$d = 3$	$d = 5$
budget	4.8M	192k	12.3M	4.8M	1.0M	6.1M	6.1M
<i>trajectory methods — positions in time order</i>							
1 binned Kramers–Moyal	0.293	0.648	0.224	0.785	1.56	0.564	1.62
2 local-linear kernel	0.126	0.358	0.088	0.280	1.25	0.261	0.925
3 SFI (Fourier basis)	0.097	0.392	0.053	0.211	1.43	0.195	0.946
4 MLP	0.100	0.340	0.080	0.319	1.38	0.223	0.927
4 CNN (one dataset)	0.115	0.380	0.068	0.333	1.44	0.224	—
4b amortised CNN	0.060	0.257	0.043	0.134	0.63	—	—
<i>snapshot methods — unordered positions, plus the true D</i>							
5 KDE score	0.130	0.132	0.130	0.843	0.14	0.231	0.983
5 denoising score matching	0.072	0.194	0.072	0.164	0.08	0.142	0.625
$\hat{b} \equiv 0$ (reference)	1	1	1	1	1	1	1

Reference: 400 walkers $\times$ 12 000 steps. Walker columns: 12 000 steps each. Position error: 256 walkers $\times$ 4000 steps, mean of 3 seeds. Dimension columns: 2048 walkers $\times$ 3000 steps. A dash: the method does not exist in that dimension (the amortised CNN is 2D only; the grid CNN is limited to $d \leq 3$).

Fourier modes, so SFI’s basis is not a convenience but a match. On fields built by other generators (sparse wells, cellular patterns) SFI loses that edge to the kernel, which assumes only smoothness. The per-dataset network sits in the middle almost everywhere and wins among the classical methods where data is scarcest (0.340 at 16 walkers).

4.2 The amortised network: a learned prior, its gain, and its price

In distribution, it wins nearly everywhere. On 100 held-out fields from the training generator, fitted at three budgets (median 64k, 256k and 1M transitions), its median nrmse is 0.213, 0.122 and 0.070 against 0.334, 0.192 and 0.089 for the better of SFI and the kernel — it is the better method on 99%, 97% and 93% of fields. On the reference walkers it reaches 0.060 against SFI’s 0.097, without cross-validation and in 0.6 seconds.

What it learned: a filter, not a bandwidth. To see what the network does, linearise it. Its output at one node c_0 responds to the mean increment $\bar{Y}_c = Y_c/N_c$ of every cell c through the *equivalent kernel*

$$K_c = \frac{\partial \hat{b}(c_0)}{\partial \bar{Y}_c}, \quad \hat{b}(c_0) \approx \sum_c K_c \bar{Y}_c, \quad (11)$$

computed exactly by automatic differentiation. Two numbers summarise it: its *width* (the RMS radius of $|K|$), and its *gain* $G = \sum_c K_c$, the fraction of a uniform drift that passes through. A classical smoother has $G = 1$ by construction (its weights sum to one) and adapts by changing its width: as data grow, the cross-validated kernel bandwidth roughly halves, from about 0.17 to 0.08. The network does the opposite:

walkers	16	64	256	1024	D	0.05	0.15	0.4	1.0
width	0.260	0.259	0.237	0.262	width	0.119	0.177	0.256	0.337
gain G	0.60	0.77	0.92	0.93	gain G	0.79	0.93	0.76	0.39

As the budget grows 64-fold its width barely moves and its gain rises from 0.60 to 0.93. As the noise grows it widens *and* shrinks harder, down to $G = 0.39$ at $D = 1$.

Why shrinking is the right thing to learn. A smoother with gain 1 is unbiased but passes all the noise it averages; multiplying the estimate by $G < 1$ adds a bias $(1 - G)b$ and cuts the variance by G^2 . When noise

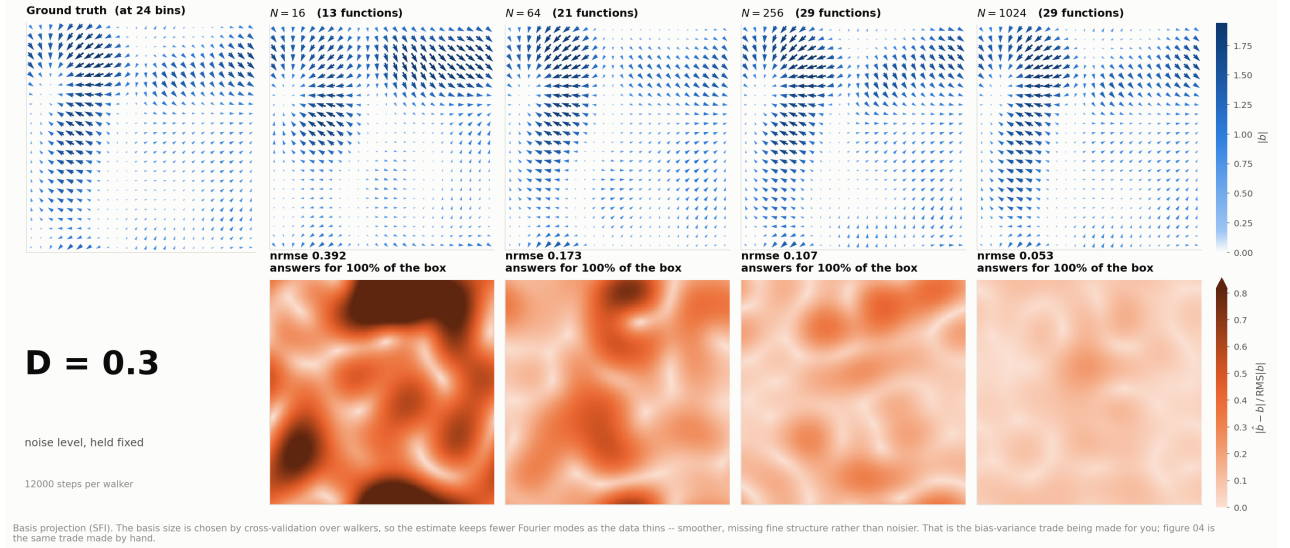


Figure 3: SFI as the data grows: ground truth, then estimates from 16 to 1024 walkers, with the error map below each. The basis size is chosen by cross-validation over walkers, so the estimate keeps fewer modes as the data thins – it goes smooth and misses fine structure rather than going noisy. Every method produces this figure on the same walkers.

dominates, that trade is strongly favourable, and the optimal amount of shrinkage depends on how large fields typically are – on the prior. In the idealised linear-Gaussian case the Bayes-optimal estimator is the Wiener filter, which acts on each Fourier mode k of the data separately,

$$\hat{b}(k) = \frac{S(k)}{S(k) + \mathcal{N}(k)} \bar{Y}(k), \quad (12)$$

with S the power spectrum of the field prior and $\mathcal{N} \propto \sigma^2/n$ the noise power per mode. More data lowers $\mathcal{N}$ and pushes the factor towards 1 – the gain rises; more noise raises it and suppresses the high- k modes first – the filter widens and shrinks. The network is not linear, and not exactly this filter, but it shows its signature. Its kernels are also elongated with negative side lobes: the x -component of a gradient field is correlated further along y than along x , and the network learned that anisotropic covariance, which no isotropic bandwidth can represent.

The price: a prior can be wrong. Everything above hinges on the training generator being a good model of the field. When it is not, the bias $(1 - G)b$ that was cheap becomes the dominant error, because the network shrinks the features its prior considers unlikely. On six generators it never saw (eight fields each, 1M transitions, same amplitude):

median nrmse	egg carton	terraces	power-law	Perlin	band-pass	sparse wells
local-linear kernel	0.283	0.602	0.612	0.642	0.442	0.375
SFI	0.163	0.619	0.729	0.743	0.379	0.356
amortised CNN	0.151	0.702	0.726	0.803	0.721	0.683
<i>amortised / best</i>	<i>0.92</i>	<i>1.17</i>	<i>1.19</i>	<i>1.25</i>	<i>1.90</i>	<i>1.92</i>

It wins only on the egg carton, a smooth single-scale field that resembles a long-correlation training field. It is worst exactly where the classical methods do well. A band-pass field concentrates its power on a ring of wavenumbers that the Gaussian prior considers improbable at that amplitude, so those modes are damped. Sparse wells – a plain of zero drift broken by narrow wells with drift up to $4\times$ the RMS – are treated as noise: smoothed away and pulled back towards the amplitudes the network knows. Shifting a parameter beyond the training range shows the same rule: a rotation $\omega = 3$ (trained $|\omega| \leq 1.5$) costs $3.3\times$, a drift twice as strong $2.2\times$, near-noise-free data ($D = 0.02$) $3.5\times$; but rougher fields, smoother fields and much noisier data, which the prior does not forbid, are survived or still won. SFI and the kernel have no training distribution to leave; their errors move only with the difficulty of the problem.

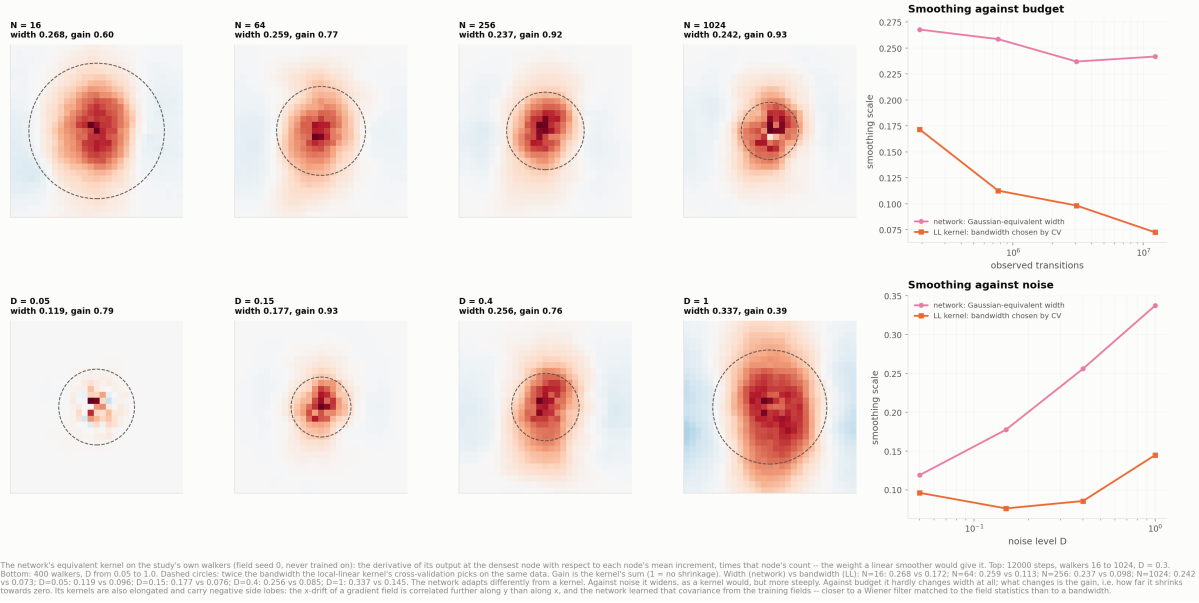


Figure 4: The network's equivalent kernel K_c on the reference field (never seen in training). Top: walkers from 16 to 1024 at $D = 0.3$. Bottom: D from 0.05 to 1.0. Dashed circles: twice the bandwidth the local-linear kernel's cross-validation picks on the same data. Gain is the kernel's sum (1 = no shrinkage). Width (network) vs bandwidth (LL): $N=16$: 0.268 vs 0.172; $N=64$: 0.259 vs 0.113; $N=256$: 0.237 vs 0.098; $N=1024$: 0.242 vs 0.073; $D=0.05$: 0.119 vs 0.096; $D=0.15$: 0.177 vs 0.076; $D=0.4$: 0.256 vs 0.085; $D=1$: 0.337 vs 0.145. The network adapts differently from a kernel. Against noise it widens, as a kernel would, but more steeply. Against budget it hardly changes width at all; what changes is the gain, i.e. how far it strays towards zero. Its kernels are also elongated and carry negative side lobes; the x-drift of a gradient field is correlated further along y than along x , and the network learned that covariance from the training fields -- closer to a Wiener filter matched to the field statistics than to a bandwidth.

The amortised network is better because it knows what the answer should look like, and it degrades exactly as far as that knowledge is wrong. Use it where the training generator is a credible model of the system; otherwise train it on a broader mix of generators.

4.3 At equilibrium, static positions beat increments, increasingly with dimension

On a gradient field ($\omega = 0$), denoising score matching on 500k independent snapshots reaches 0.072 in 2D, where the best per-dataset trajectory method on 4.8M increments gets 0.097. Using the walkers' own (time-correlated) positions instead of idealised snapshots, it still beats every per-dataset trajectory method at 16 and 64 walkers (0.302 and 0.164, against 0.340 and 0.173 for the best of them). The advantage grows with dimension:

	trajectories, 6.1M transitions					snapshots, up to 500k positions			
	$d = 2$	3	5	10		$d = 2$	3	5	10
SFI	0.082	0.195	0.946	1.002	KDE score	0.128	0.231	0.983	0.997
local-linear	0.116	0.261	0.925	1.004	DSM	0.072	0.142	0.625	0.996
MLP	0.093	0.223	0.927	1.019					

At $d = 5$ every trajectory method sits near 0.93, barely better than predicting zero, while denoising score matching reaches 0.625 -- from twelve times fewer samples. Three things explain it.

Increments lose information as d grows; positions do not. Information about b accumulates with observed *time*, not with the number of steps: the variance of a local estimate is $2D/T_{\text{occ}}$, with T_{occ} the time spent in the region (§5). A stable simulation needs Δt to shrink with d (the diffusive step $\sqrt{2dD\Delta t}$ must stay small against the field's length scale): $\Delta t = 1.05 \times 10^{-3}$ in 2D but 2.7×10^{-4} in 5D. The same 6.1M transitions therefore cover four times less time, and the per-step noise grows from 32 to 109 times the drift. Meanwhile the field has ~ 1700 independent correlation volumes in 5D. The 1600 time units of data leave about one unit per volume, a standard deviation $\sqrt{2D} \approx 0.8$ per component against a signal of 0.44: below one in signal-to-noise everywhere, which is why every trajectory method retreats towards zero. Not capacity -- the same MLP fits the exact 5D drift to 0.04 given noiseless targets -- but information.

A snapshot is a full, independent sample. Each position is a draw from ρ_{ss} , whose log-gradient is the drift (divided by D); its information does not depend on Δt at all. 500k snapshots give ~ 300 independent samples per correlation volume, and the noise in the score regression is the noise level σ the method chooses, not the $\sqrt{2D/\Delta t}$ imposed by the physics.

A shared model, not a local average. The KDE score is a fixed-bandwidth local average whose gradient variance grows like $h^{-(d+2)}$ — the same curse as binning, and it fails in 5D (0.983). Denoising score matching fits one neural network across the whole space, pooling statistical strength between regions. At $d = 10$, 500k snapshots are no longer enough for either.

This advantage lives entirely on the equilibrium side of (6): it requires $J_{ss} = 0$, and it disappears with the circulation, as the next section shows.

4.4 Snapshots are blind to circulation, exactly

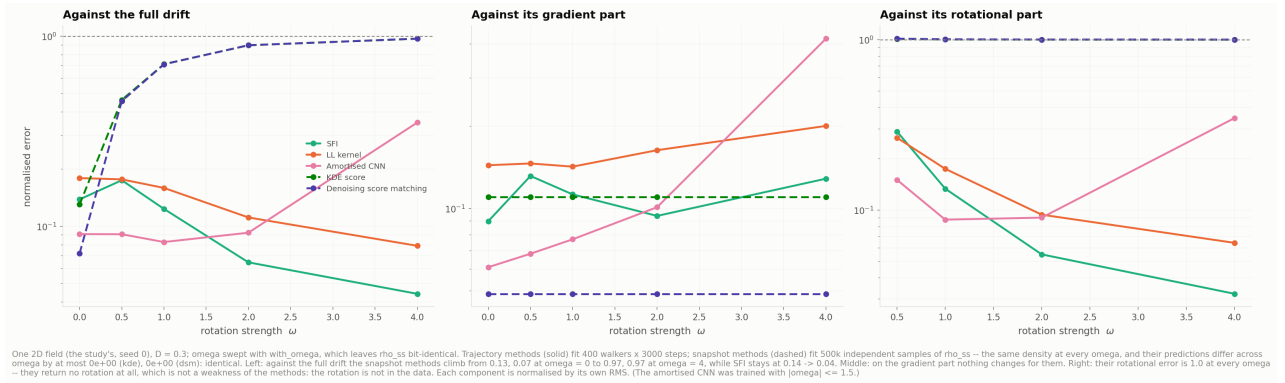


Figure 5: One field with ω swept from 0 to 4, which leaves ρ_{ss} bit-identical. Left: against the full drift the snapshot methods (dashed) climb towards 1 while trajectory methods improve. Middle: against the gradient part the snapshot methods are flat. Right: against the rotational part their error is 1.00 everywhere — they return no rotation at all.

As (6) predicts, both snapshot estimators return *bit-identical* predictions for $\omega = 0, \dots, 4$ (largest difference across the five values: exactly 0.0). Their error against the rotational component is 1.00 at every ω , and against the full drift it climbs as $\omega/\sqrt{1+\omega^2}$, the fraction of $|b|$ that is rotational. Trajectory methods instead *improve* with ω : a stronger rotation is more signal against the same noise. The amortised network tracks ω up to 2 and fails at 4, outside its training range.

4.5 Position measurement error: a bias for trajectories, almost nothing for snapshots

In experiments positions are measured with error — for instance the localisation error of a microscope. Model it as $\tilde{x}_k = x_k + \varepsilon_k$ with independent $\varepsilon_k \sim \mathcal{N}(0, \sigma^2 I)$. The two families react completely differently.

Trajectories. The measured increment is $\Delta \tilde{x}_k = \Delta x_k + \varepsilon_{k+1} - \varepsilon_k$. This adds variance $2\sigma^2$, which averages out like any other noise. It also adds a *bias*, which does not. Every trajectory method groups increments by their measured starting point $\tilde{x}_k$, and the same ε_k that displaced that measurement enters the increment with the opposite sign. Suppose ρ_{ss} increases to the right. A particle measured at $\tilde{x}$ is more likely to truly sit to its right, where there are more particles, so the error $\varepsilon_k = \tilde{x}_k - x_k$ tends to point left; precisely, for small σ (Tweedie's formula),

$$\mathbb{E}[\varepsilon_k | \tilde{x}_k] = -\sigma^2 \nabla \log \rho_{ss}(\tilde{x}_k). \quad (13)$$

The increment carries $-\varepsilon_k$, which therefore points right, towards higher density — along b . Dividing by Δt and using $\nabla \log \rho_{ss} = b/D$:

$$\mathbb{E}\left[\frac{\Delta \tilde{x}_k}{\Delta t} \middle| \tilde{x}_k\right] \approx b + \frac{\sigma^2}{\Delta t} \nabla \log \rho_{ss} = b \left(1 + \frac{\sigma^2}{D\Delta t}\right). \quad (14)$$

Every trajectory method then estimates a field that is too large by this factor, however much data it has. The ratio compares the measurement error σ with the size of one diffusive step $\sqrt{2D\Delta t} \approx 0.025$: at $\sigma = 0.02$,

one percent of the box, they are comparable, and the estimate is $2.3\times$ too large. Recording less often (larger Δt) helps; more data does not.

Snapshots. Noisy positions are exact samples of the blurred density $\rho_{\text{ss}} * \mathcal{N}(0, \sigma^2 I)$, whose score differs from the true one by a relative amount of order $(\sigma/\ell)^2 \approx 10^{-2}$, with $\ell \approx 0.24$ the length scale of the field.

σ_{obs}	0	0.005	0.01	0.02
predicted inflation $1 + \sigma^2/(D\Delta t)$	1	1.08	1.33	2.31
binned	0.473	0.499	0.662	1.562
local-linear kernel	0.222	0.220	0.350	1.246
SFI	0.200	0.237	0.439	1.431
MLP	0.212	0.238	0.422	1.379
amortised CNN	0.140	0.153	0.240	0.627
KDE score	0.135	0.136	0.137	0.140
denoising score matching	0.077	0.077	0.078	0.080

nmse, 256 walkers $\times$ 4000 steps, $D = 0.3$, mean of 3 seeds.

The trajectory errors track the predicted inflation: an estimate $2.3\times$ too large has an nmse of at least 1.3 from the bias alone, and the classical methods land between 1.25 and 1.56. The amortised network suffers less only by accident: its amplitude prior pulls the inflated estimate back towards field magnitudes it has seen. The snapshot methods barely move. The full derivation, verified against simulation term by term, is in the companion note docs/binning_km_error_analysis.pdf.

4.6 Every method that tunes itself can be fooled by its own criterion

Four instances, each caught only by comparing against the truth. Cross-validation chose a 10^6 -cell grid that answered for 10% of the box (binning). Early stopping quit 30% early on a flat held-out curve (MLP). The KDE bandwidth search ran to the bottom of its grid on temporally correlated positions (1.06 against 0.35 at a fixed bandwidth), because it split samples at random rather than by walker; and to the *top* at $D = 1$, where the density is nearly flat and the criterion has almost no signal – the 0.843 in the table. Every method therefore reports what it chose, not only what it scored.

5 The error budget, in closed form

For binning the whole error decomposes analytically, and each term was verified against simulation. For a cell of width h holding n increments,

$$\text{MSE} \simeq \underbrace{\frac{2D}{n\Delta t}}_{\text{variance}} + \underbrace{\frac{|\nabla b|^2 h^2}{12}}_{\text{bias: one value per cell}} + \underbrace{b^2 \left(\frac{\sigma_{\text{obs}}^2}{D\Delta t} \right)^2}_{\text{bias: position error}}, \quad n \simeq M \rho_{\text{ss}} h^d, \quad (15)$$

with M the total number of transitions. Substituting n gives three facts that govern every trajectory method:

- $\text{Var} = 2D/(M \rho_{\text{ss}} h^d \Delta t) = 2D/T_{\text{occ}}$ depends only on the *time* spent in the cell. Sampling faster makes each $\Delta x/\Delta t$ noisier and gives proportionally more of them: Δt cancels.
- Balancing variance against bias gives $h^* \propto M^{-1/(d+2)}$ and $\text{MSE}^* \propto M^{-2/(d+2)}$: to halve the error at $d = 6$ takes $16\times$ the data. This is the basic reason to go beyond binning.
- The position-error term keeps its $1/\Delta t$, so with imperfect measurements sampling faster is actively harmful.

Against 64 independent simulated replicas: the variance law holds to within 1–9% at every cell width and local density, the predicted optimal bin count is 17.0 against a measured 16, and the inflation factor is confirmed to 1.5%.

6 Limitations

- **The families are not compared like for like.** The amortised network knows the field distribution; the snapshot methods get independent samples and the true D .
- **One field family, one box.** Almost every number comes from Gaussian-spectrum fields on $[-1, 1]^2$; the other-generator test is the only breadth check, and it is 2D.
- D is known to the snapshot methods, and Δt is set by the simulator's stability rule rather than by an experiment.
- **The snapshot estimators receive positions without walker labels**, so their cross-validation cannot group by walker. That is a real defect when positions are correlated in time.
- **The amortised network is 2D-only**, and its 64^2 output grid caps its accuracy on nearly noise-free data.

7 What is worth doing next

1. **A hybrid estimator:** the gradient part from positions, the rotational remainder from increments. With few walkers it should beat both families.
2. **Estimate D from the data** (quadratic variation) in the snapshot methods and report how the drift scale degrades.
3. **Train the amortised network on a mixture of generators** and repeat the test above; how much in-family accuracy it trades for robustness is the interesting number.
4. **A calibrated posterior:** a conditional diffusion model over drift fields giving a distribution rather than a point estimate, with coverage checked against the truth.
5. **A time-resolution sweep**, where the error budget predicts the $O(\Delta t)$ discretisation bias trading against the $1/\Delta t$ measurement-error amplification.
6. **Real data** — single-particle tracking or an ecological time series: unknown D , non-stationary boundaries, heterogeneous noise.

8 Reproducing this

Every figure and table comes from a numbered script: 01 (fields and trajectories), 03 (the six shared figures for every method), 04 (kernel against binning), 05–07 (amortised network: data, training, study), 08 (snapshot methods and identifiability), 09 (the error budget); `python -m pytest tests/` runs the 46 tests. Result tables are cached by configuration in `outputs/*/data/`, so a re-run redraws the figures without recomputing anything. Per-method detail is in the `README.md` of each folder under `outputs/`, the cross-method comparison in `outputs/SUMMARY.md`, and the framework reference and the analytical note in `docs/`.